```
// ---------- Storage keys & state ----------
const TX_KEY  = "pet_transactions";
const CAT_KEY = "pet_categories";

let transactions = JSON.parse(localStorage.getItem(TX_KEY)  || "[]");
let categories   = JSON.parse(localStorage.getItem(CAT_KEY) ||
    '["Food","Travel","Shopping","Bills","Salary","Education"]');

// ---------- DOM references ----------
const $ = (id) => document.getElementById(id);
const form = $("txForm");

// ---------- Helpers ----------
const money = (n) => "₹" + n.toLocaleString("en-IN", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
const save  = () => {
  localStorage.setItem(TX_KEY, JSON.stringify(transactions));
  localStorage.setItem(CAT_KEY, JSON.stringify(categories));
};

// ---------- Categories ----------
function renderCategories() {
  const options = categories.map((c) => `<option value="${c}">${c}</option>`).join("");
  const current = $("category").value;
  $("category").innerHTML = options;
  if (categories.includes(current)) $("category").value = current;

  const fcur = $("filterCategory").value || "all";
  $("filterCategory").innerHTML = '<option value="all">All Categories</option>' + options;
  $("filterCategory").value = categories.includes(fcur) ? fcur : "all";

  $("catList").innerHTML = categories
    .map((c) => `<li>${c}<button title="Delete" onclick="deleteCategory('${c}')">×</button></li>`)
    .join("");
}

$("addCatBtn").addEventListener("click", () => {
  const name = $("newCategory").value.trim();
  if (!name) return;
  if (categories.some((c) => c.toLowerCase() === name.toLowerCase())) {
    alert("Category already exists.");
    return;
  }
  categories.push(name);
  $("newCategory").value = "";
  save();
  renderCategories();
});

function deleteCategory(name) {
  if (transactions.some((t) => t.category === name)) {
    alert("This category is used in transactions and cannot be deleted.");
    return;
  }
  categories = categories.filter((c) => c !== name);
  save();
  renderCategories();
}

// ---------- Add / Edit / Delete transactions ----------
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const amount = parseFloat($("amount").value);
  if (!(amount > 0)) { alert("Enter a valid amount."); return; }

  const data = {
    id: $("txId").value || String(Date.now()),
    type: $("type").value,
    amount,
    date: $("date").value,
    category: $("category").value,
    description: $("description").value.trim(),
  };

  const idx = transactions.findIndex((t) => t.id === data.id);
  if (idx >= 0) transactions[idx] = data; else transactions.push(data);
```

```
  save();
  resetForm();
  refresh();
});

function editTransaction(id) {
  const t = transactions.find((x) => x.id === id);
  if (!t) return;
  $("txId").value = t.id;
  $("type").value = t.type;
  $("amount").value = t.amount;
  $("date").value = t.date;
  $("category").value = t.category;
  $("description").value = t.description;
  $("formTitle").textContent = "Edit Transaction";
  $("saveBtn").textContent = "Update Transaction";
  $("cancelBtn").hidden = false;
  window.scrollTo({ top: 0, behavior: "smooth" });
}

function deleteTransaction(id) {
  if (!confirm("Delete this transaction?")) return;
  transactions = transactions.filter((t) => t.id !== id);
  save();
  refresh();
}

function resetForm() {
  form.reset();
  $("txId").value = "";
  $("date").value = new Date().toISOString().slice(0, 10);
  $("formTitle").textContent = "Add Transaction";
  $("saveBtn").textContent = "Add Transaction";
  $("cancelBtn").hidden = true;
}
$("cancelBtn").addEventListener("click", resetForm);

// ---------- Search & filter ----------
function getFiltered() {
  const q    = $("search").value.trim().toLowerCase();
  const cat  = $("filterCategory").value;
  const type = $("filterType").value;
  const from = $("fromDate").value;
  const to   = $("toDate").value;

  return transactions
    .filter((t) =>
      (!q || t.description.toLowerCase().includes(q) || t.category.toLowerCase().includes(q)) &&
      (cat === "all" || t.category === cat) &&
      (type === "all" || t.type === type) &&
      (!from || t.date >= from) &&
      (!to || t.date <= to)
    )
    .sort((a, b) => b.date.localeCompare(a.date));
}

["search", "filterCategory", "filterType", "fromDate", "toDate"].forEach((id) =>
  $(id).addEventListener("input", renderTable)
);
$("clearFilters").addEventListener("click", () => {
  $("search").value = ""; $("filterCategory").value = "all"; $("filterType").value = "all";
  $("fromDate").value = ""; $("toDate").value = "";
  renderTable();
});

// ---------- Render table ----------
function renderTable() {
  const list = getFiltered();
  $("txBody").innerHTML = list.map((t) => `
    <tr>
      <td>${t.date}</td>
      <td>${t.description}</td>
      <td>${t.category}</td>
      <td><span class="badge ${t.type}">${t.type === "income" ? "Income" : "Expense"}</span></td>
      <td class="${t.type === "income" ? "inc" : "exp"}">${t.type === "income" ? "+" :
          "-"}${money(t.amount)}</td>
```

```
      <td class="r">
        <button class="act edit" onclick="editTransaction('${t.id}')">Edit</button>
        <button class="act del" onclick="deleteTransaction('${t.id}')">Delete</button>
      </td>
    </tr>`).join("");
  $("emptyMsg").hidden = list.length > 0;
}

// ---------- Totals (calculated dynamically) ----------
function renderTotals() {
  const inc = transactions.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0);
  const exp = transactions.filter((t) => t.type === "expense").reduce((s, t) => s + t.amount, 0);
  $("totalIncome").textContent  = money(inc);
  $("totalExpense").textContent = money(exp);
  $("balance").textContent      = money(inc - exp);
  $("balance").style.color      = inc - exp >= 0 ? "#16a34a" : "#dc2626";
}

// ---------- Monthly summary ----------
function monthlyData() {
  const map = {};
  transactions.forEach((t) => {
    const m = t.date.slice(0, 7);
    map[m] = map[m] || { income: 0, expense: 0 };
    map[m][t.type] += t.amount;
  });
  return Object.keys(map).sort().map((m) => ({ month: m, ...map[m] }));
}

function monthLabel(ym) {
  const [y, m] = ym.split("-");
  return new Date(y, m - 1).toLocaleString("en-US", { month: "short", year: "numeric" });
}

function renderMonthly() {
  const rows = monthlyData();
  $("monthBody").innerHTML = rows.length
    ? rows.map((r) => `
      <tr>
        <td>${monthLabel(r.month)}</td>
        <td class="inc">${money(r.income)}</td>
        <td class="exp">${money(r.expense)}</td>
        <td class="r"><strong>${money(r.income - r.expense)}</strong></td>
      </tr>`).join("")
    : '<tr><td colspan="4" class="empty">No data yet.</td></tr>';
}

// ---------- Chart (Canvas bar chart) ----------
function chartData() {
  if ($("chartMode").value === "month") {
    return monthlyData().map((r) => ({ label: monthLabel(r.month), value: r.expense }));
  }
  const map = {};
  transactions.filter((t) => t.type === "expense").forEach((t) => {
    map[t.category] = (map[t.category] || 0) + t.amount;
  });
  return Object.entries(map).map(([label, value]) => ({ label, value }));
}

function renderChart() {
  const canvas = $("chart");
  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);

  const data = chartData();
  if (!data.length) {
    ctx.fillStyle = "#64748b"; ctx.font = "14px Segoe UI, Arial"; ctx.textAlign = "center";
    ctx.fillText("No expense data to display", W / 2, H / 2);
    return;
  }

  const colors = ["#2563eb", "#f59e0b", "#16a34a", "#dc2626", "#8b5cf6", "#0891b2", "#db2777"];
  const pad = { l: 55, r: 15, t: 20, b: 45 };
  const max = Math.max(...data.map((d) => d.value)) * 1.15;
  const chartH = H - pad.t - pad.b;
```

```
  const chartW = W - pad.l - pad.r;
  const bw = Math.min(60, chartW / data.length * 0.6);

  // grid lines + y labels
  ctx.font = "11px Segoe UI, Arial"; ctx.textAlign = "right"; ctx.strokeStyle = "#e2e8f0";
  for (let i = 0; i <= 4; i++) {
    const y = pad.t + chartH - (chartH * i) / 4;
    ctx.beginPath(); ctx.moveTo(pad.l, y); ctx.lineTo(W - pad.r, y); ctx.stroke();
    ctx.fillStyle = "#64748b";
    ctx.fillText("₹" + Math.round((max * i) / 4), pad.l - 6, y + 4);
  }

  // bars
  data.forEach((d, i) => {
    const slot = chartW / data.length;
    const x = pad.l + slot * i + (slot - bw) / 2;
    const h = (d.value / max) * chartH;
    const y = pad.t + chartH - h;
    ctx.fillStyle = colors[i % colors.length];
    ctx.fillRect(x, y, bw, h);

    ctx.fillStyle = "#1e293b"; ctx.textAlign = "center";
    ctx.fillText("₹" + Math.round(d.value), x + bw / 2, y - 5);
    ctx.fillStyle = "#64748b";
    ctx.fillText(d.label, x + bw / 2, H - pad.b + 16);
  });
}
$("chartMode").addEventListener("change", renderChart);

// ---------- Refresh everything when records change ----------
function refresh() {
  renderTotals();
  renderTable();
  renderMonthly();
  renderChart();
}

// ---------- Init ----------
renderCategories();
resetForm();
refresh();
```